

ClientOpsCRM Research and Generative Engineering Framework

Research objective

The goal was not to clone a vendor screen or reproduce a fashionable feature list. The research question was: which CRM primitives remain structurally important across enterprise products, smaller sales CRMs, academic CRM theory, data-system engineering and secure software practice, and how can those primitives be implemented with deterministic local behavior on top of Relay's existing intake core?

The release research corpus contains 143 unique URLs: 77 inherited from Relay 1.2.1 and 66 added for this CRM work. It includes first-party product documentation, database/security standards, peer-reviewed CRM and decision-science work, and an official Nobel Prize source. The canonical list is `research/research_sources.csv`; design claims are mapped in `research/CLAIM_SOURCE_LEDGER.csv`.

Cross-source convergence

CRM is a process system, not a contact list

Payne and Frow's strategic framework treats CRM as cross-functional processes spanning strategy, value creation, multichannel integration, information management and performance assessment. Reinartz, Krafft and Hoyer operationalize CRM as a measurable process rather than a single software feature. Kumar and Reinartz similarly separate strategic, analytical and operational CRM. This argues for a small coherent state machine with measurable transitions rather than isolated address-book CRUD.

Lead and opportunity are different states

Dynamics 365, Pipedrive, Odoo and SAP all distinguish earlier lead/qualification work from opportunities/deals that enter a sales cycle or pipeline. The release therefore does not put every intake record directly into the opportunity pipeline. `lead_qualify` is explicit and `lead_convert` is transactional.

Relationships need durable entity identity

HubSpot's current contacts/companies/deals APIs emphasize separate objects linked by associations and unique identifiers. Dynamics models accounts and contacts separately; Pipedrive distinguishes people and organizations. ClientOpsCRM mirrors the durable concept, not vendor schemas: accounts and contacts are independent records, and deals link to them without embedding copies as the only source of truth.

Pipeline state must be explicit

Pipedrive documents stages as steps through a real sales process and uses stage/deal probability for pipeline management. Odoo and SAP use stage/probability/value for pipeline analysis and forecasting. ClientOpsCRM stores the pipeline, stage, stage kind and probability explicitly and checks their consistency below the command layer.

Activities are first-class history

Pipedrive, Odoo and SAP all associate calls/tasks/meetings/follow-up activities with CRM objects. ClientOpsCRM supplies tasks plus a generic business activities history so state changes do

not disappear into the current record snapshot.

Audit is different from activity history

Microsoft Dataverse auditing exists to answer who changed what and when. This is a different purpose from the sales timeline. ClientOpsCRM therefore has both activities and audit_log; audit is guarded against update/delete in SQLite.

Search, custom fields and import are core extensibility surfaces

HubSpot has dedicated search, properties and imports APIs; Pipedrive provides data/custom fields and spreadsheet import. ClientOpsCRM makes FTS search, custom values and CSV/JSON interchange first-class rather than treating them as later decoration.

Forecasts should expose assumptions

SAP/Odoo/Pipedrive all connect opportunity values, probability/stage and expected closing to forecasts or pipeline analysis. Decision-science research by Tversky and Kahneman warns that judgments under uncertainty are framing-sensitive. ClientOpsCRM therefore deliberately labels its forecast as deterministic weighted pipeline arithmetic, not an objective probability model or AI prediction. The operator can inspect every value and stage probability used.

Engineering framework used

The implementation loop is evidence-driven:

1. **Observe** — inspect the real Relay files and executable behavior.
2. **Extract invariants** — identify durable concepts repeated across credible sources.
3. **Specify failure before success** — define invalid transitions, duplicate identities, malformed data, partial-write hazards and audit tampering before coding happy paths.
4. **Implement at two layers** — command validation plus database constraints/triggers for invariants that must survive direct SQL access.
5. **Construct inverse/adversarial tests** — attempt precisely the mutation or partial state the invariant is meant to reject.
6. **Run build parity** — release/debug/sanitizer builds use the same source and warning policy.
7. **Preserve provenance** — actor, source lead, Relay UUID/metadata, activities and audit survive transformations.
8. **Package only proven artifacts** — generated release bytes are hashed, enumerated and ZIP-tested.

This is the project's “new-gen” framework: novelty is not an unsupported feature. Novelty is produced by recombining proven CRM process structure with Relay's deterministic intake, transactional local persistence, executable invariants, and adversarial verification so the resulting system has a smaller trust surface than a typical multi-service CRM stack.

Design decisions that were rejected

- Copying vendor schemas or branding: unnecessary and would reduce independence.
- Treating all Relay requests as deals: destroys the qualification boundary.
- Floating-point money: avoidable error source.
- Free-form deal status independent of stage: permits contradictory states.
- Mutable audit records: defeats the purpose of local audit evidence.
- Regex-heavy validation solely for appearance: deterministic parsers are easier to audit and avoided a compiler/sanitizer false-positive encountered during strict builds.
- Silently replacing an unknown schema: risks corruption/downgrade.

- Generating an AI forecast without a validated training/evaluation dataset: would be a mock prediction rather than evidence-backed functionality.

Research stop rule

Research stopped after the core design claims were independently supported across multiple product families or primary technical standards, academic CRM sources supplied the process-level model, the added corpus exceeded the requested 100-source gate, and additional vendor pages were no longer changing the entity/state/integrity design. The retained corpus remains available for future feature work.